

Slack Variables for Non Linearly Separable Case

© Copyright 2021 Fusemachines, Inc. | All rights reserved

Prerequisites

Students must know of the following topics:

- Margin, Support vectors
- Linear separable case

Learning Objectives

After reading this notebook, students will be able to:

- Know the concepts of linearly Non Separable Data
- Transform features to another space
- Slack Variables Equation

Everything that we have done so far has assumed the data is in the linearly separable case how we maximize the margin and also some optimization problem.

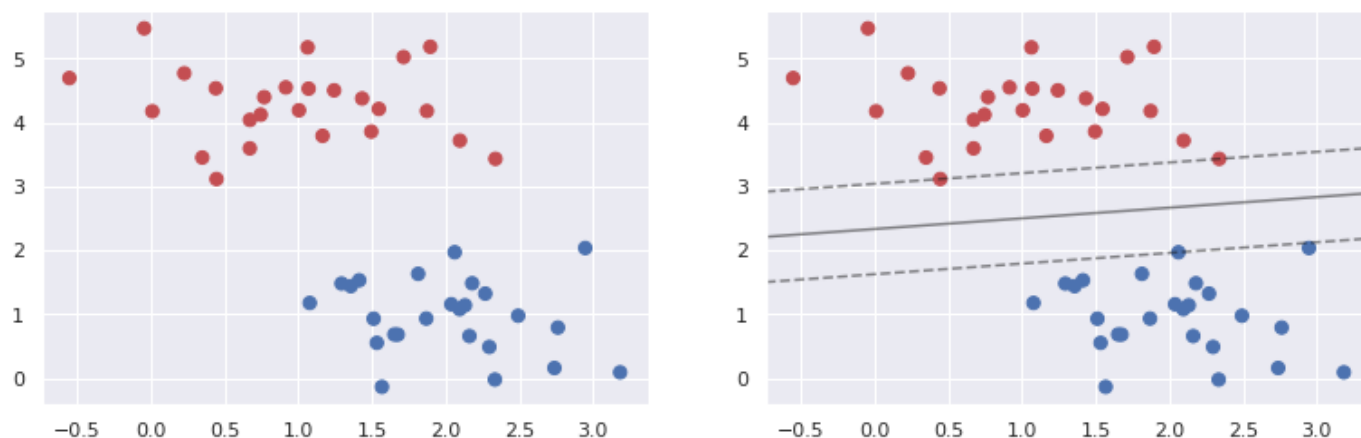


Figure 1: Linearly Separable Datapoints: (Left) Scatter data points, (Right) data points with hyperplane

The above figure shows linearly separable data, which are depicted in the 2D space. Let's see what we should do in the case of a non-linear separable dataset?

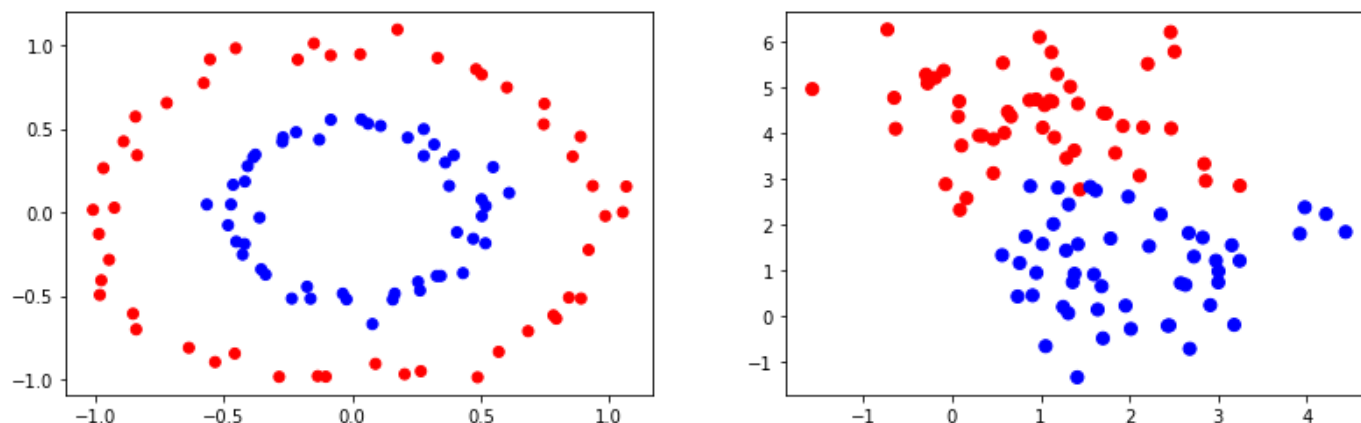


Figure 2: Scatter Binary Datapoints: (Left) Non linear data points, (Right) In-separable data points

Let's dive into possible solutions.

1. Feature map / Kernel: to map data to a higher dimension and make linearly separable.
2. Softening the separation: using a slack variable to penalize the few misclassifications.

The figure below shows how the non linear data(circular data points) in 2D space change to linearly separable by mapping to a higher dimension, i.e., 3D space.

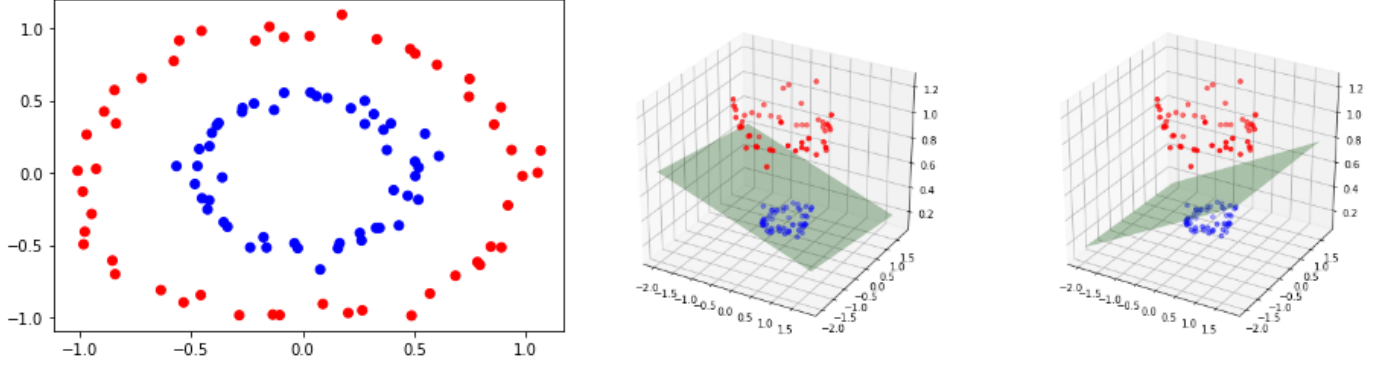


Figure 3: Plot Non linear Datapoints: (1) data points in 2D, (2 & 3) Plot in Higher dimension

Note: Please visit the chapter kernel to learn more about its working mechanism.

In this chapter, we will discuss the nonlinear separable case using slack variables. Before that, let's recall the equation of maximum margin classifier. i.e.,

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad (1)$$

subject to:

$$y_i(\mathbf{w}^T \cdot x_i + b) \geq 1, \quad i = 1, \dots, n$$

In the Figure (3), we have circular datapoint in left and inseparable data points in right. In right figure we misclassify a single datapoint. The data is most like to be on the opposite side of its group. We can modify the SVM algorithm we studied in the previous chapter to suit our need for better hyperplane even when the two classes' data are overlap. In most cases, the overlapping of the class is due to the misclassification of data.

To solve the problem of suboptimal hyperplane in the case of mislabeled data, we introduce a slack variable $\xi = \{\xi_1, \xi_2, \dots, \xi_n\}$ and modify the constraints as,

$$\begin{aligned} y_i(\mathbf{w}^T \cdot x_i + b) &\geq 1 - \xi_i, \quad i = 1, \dots, n \\ \xi_i &\geq 0, \quad i = 1, \dots, n \end{aligned} \quad (2)$$

The value of ξ_i in $y_i(\mathbf{w}^T x_i + b) \geq 1 - \xi_i$ is the proportional amount by which the prediction $\mathbf{w}^T x_i + b$ is on the wrong side of the margin. The value of ξ_i is more than one when x_i is being misclassified. Now we add the term into minimization problem so that we minimize $\mathbf{w}^T + C * (\text{a distance of misclassified points from the current line})$. that means the optimization formula becomes,

$$\min_{\mathbf{w}, b} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^n \xi_i \quad (3)$$

Also,

$$\xi_i = \begin{cases} 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) & \text{if } y_i(\mathbf{w}^\top \mathbf{x}_i + b) < 1 \\ 0 & \text{if } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \end{cases} \quad (4)$$

The parameter C in equation (3) is known as cost parameter and is the hyperparameter of SVM. In above equation (4), the value of ξ_i is zero when the point $\mathbf{x}_i$ is correctly classified by the model and also is not inside the margin or far from margin (*i.e.* $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$) and has some value when we miss classifying the data.

Equation (3) can be written as,

$$\min_{\mathbf{w}, b} \mathbf{w}^\top \mathbf{w} + C \sum_{i=1}^n \max(1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b), 0) \quad (5)$$

Equation (5) is the function that we are trying to minimize with respect to $\mathbf{w}$. We can use algorithms like gradient descent to optimize this function. The terms in the equation (5) can be categorized into two parts: first, the regularization term, and second, the loss function.

$$\begin{aligned} \text{Regularization, } \mathcal{R}_{\mathbf{w}} &= \min_{\mathbf{w}, b} \mathbf{w}^\top \mathbf{w} \\ \text{Loss, } \mathcal{L}_{\mathbf{w}}(x, y) &= C \sum_{i=1}^n \max(1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b), 0) \end{aligned}$$

The cost of our learning algorithm can be written as,

$$\text{Cost}_{\mathbf{w}}(x, y) = \mathcal{L}_{\mathbf{w}}(x, y) + \gamma \mathcal{R}_{\mathbf{w}} \quad (6)$$

In the equation, (6), γ is the regularization parameter and is the hyperparameter of our learning algorithm.

The `make_classification` function from the `sklearn` is used to generate artificial data of two classes. We will then fit different SVM models with varying values of cost parameter C .

The lower value of cost parameter, the larger the margin, and vice-versa. This behavior is due to the fact that when we have a lower value of C , having many ξ_i non-zero will not be costly when optimizing. The loss is directly proportional to $C \cdot \xi_i$. The loss will increase

- if any of the value of ξ_i increases keeping C constant.
- if C increases, keeping all the ξ_i constant.

After all, the algorithm has to pay for each of them. The algorithm now has to choose whether to allow slack to increase the margin or decrease its regularization. Or not allowing slack, have a small margin with a large value of regularization term.

In []:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from sklearn.datasets import make_classification

# random_state=20 -> gives dataset with few point very near each other (good example)
# random_state=25 -> gives inseperable case, so can show nice separating line some
X,y = make_classification(n_samples=20, n_features=2, n_redundant=0, n_repeated=0,
                        n_classes=2, n_clusters_per_class=1, random_state=20, wei

y = y * 2 - 1

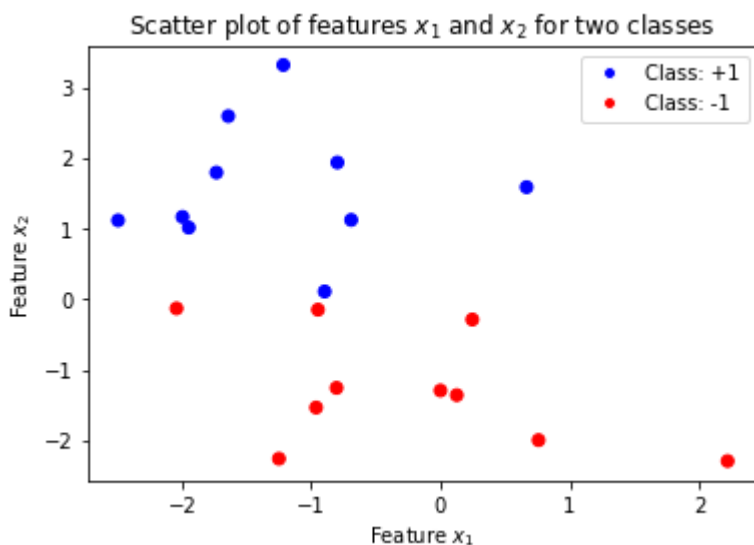
scatter = plt.scatter(X[:, 0], X[:, 1], c=['r' if _y < 0 else 'b' for _y in y])
plt.title("Scatter plot of features $x_1$ and $x_2$ for two classes")
plt.ylabel("Feature $x_2$")
plt.xlabel("Feature $x_1$")

lp = Line2D(range(2), range(1), color="white", marker='o', markerfacecolor="blue")
ln = Line2D(range(2), range(1), color="white", marker='o', markerfacecolor="red")

plt.legend(handles=(lp, ln), labels=("Class: +1", "Class: -1"))
```

Out[7]:

<matplotlib.legend.Legend at 0x7f7a71ed6f60>



In []:

```
def train_svm(Xin, yin, C, gamma, lr=0.01):
    np.random.seed(17)

    theta = np.random.randn(Xin.shape[1])
    b = np.random.rand()

    N = X.shape[0]
    learning_rate = 0.01

    epochs = 1000
    for epoch in range(epochs):
        dtheta = np.zeros_like(theta)
        db = 0
        for i in range(N):
            pred = np.dot(theta.T, X[i]) + b
            if (yin[i] * pred) < 1:
                dtheta = dtheta - np.dot(yin[i], X[i].T)
                db = db - yin[i]

        # multiply the gradient from loss term by C and add the gradient
        # from regularization term
        dtheta = C * dtheta + 2 * gamma * theta
        db = C * db

        # update the parameters
        theta = theta - learning_rate * dtheta
        b = b - learning_rate * db

        # if (epoch + 1) % 500 == 0:
        #     print(f'epoch {epoch}, dtheta: {dtheta}, db: {db}, theta: {theta}, b:
    return theta, b
```

In []:

```
def plot_hyperplane(Xin, yin, theta, b):
    plt.scatter(Xin[:, 0], Xin[:, 1], c=['r' if _y < 0 else 'b' for _y in y])
    xmin, xmax = -5, 5
    ymin, ymax = -5, 5
    npoints = 500

    _xm = np.linspace(xmin, xmax, npoints)
    _ym = np.linspace(ymin, ymax, npoints)

    color_margin = '#3385ff'

    Xs, Ys = [], []
    colors = []

    for _x in _xm:
        for _y in _ym:
            v = np.dot(theta.T, np.array([[_x], [_y]])) + b
            if v >= -1 and v <= 1:
                Xs.append(_x)
                Ys.append(_y)
                colors.append(color_margin)

    plt.scatter(Xs, Ys, c=colors, alpha=0.3)
    plt.axis([xmin, xmax, ymin, ymax])

    _x = np.linspace(xmin, xmax, npoints)
    # get value of y for every _x from optimal hyperplane
    # defined by 'theta' and 'b'
    _y = - (_x * theta[0] + b) / theta[1]

    # hyperplane at the positive side, np.dot(theta.T, x) = +1
    _yp = (1 - _x * theta[0] - b) / theta[1]

    # hyperplane at the negative side, np.dot(theta.T, x) = -1
    _yn = (-1 - _x * theta[0] - b) / theta[1]

    plt.scatter(X[:, 0], X[:, 1], c=['r' if _y < 1 else 'b' for _y in y])
    plt.plot(_x, _y, c='r')
    plt.plot(_x, _yn, c='y', alpha=0.8)
    plt.plot(_x, _yp, c='y', alpha=0.8)
```

In []:

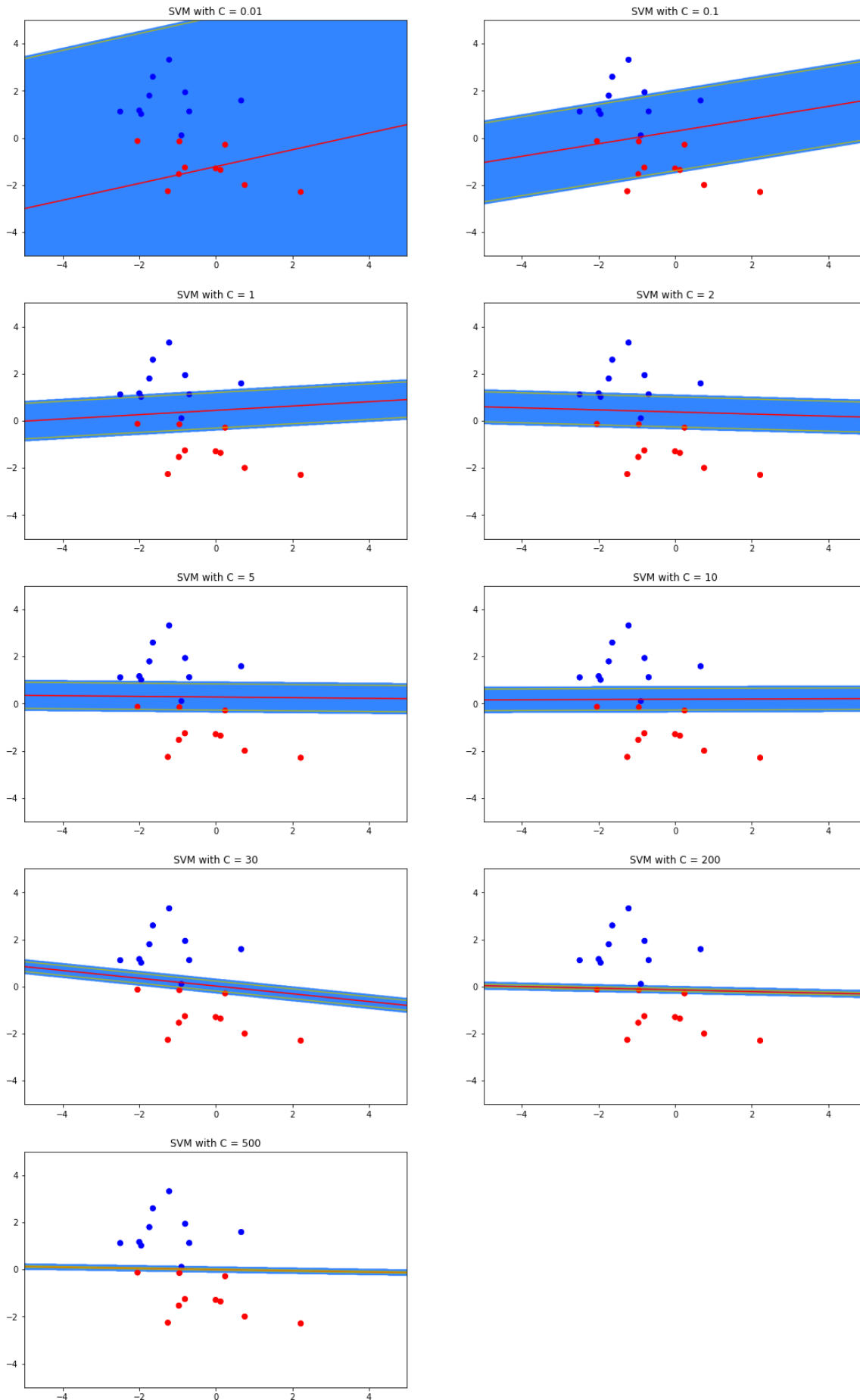
```
C = [0.01, 0.1, 1, 2, 5, 10, 30, 200, 500]
gamma = 0.9

plt.figure(figsize=(18,30))
plt.rcParams['figure.facecolor'] = 'white'

ncols = 2
nrows = np.ceil(len(C)/ncols)

for i, _C in enumerate(C):
    theta, b = train_svm(X, y, _C, gamma)
    plt.subplot(nrows, ncols, i+1)
    plot_hyperplane(X, y, theta, b)
    plt.title(f'SVM with C = {_C}')

plt.show()
```



From the above plots, we can see that when the value of C increases, the margin decreases.

For a large enough value of C ($C \rightarrow \infty$), it tries to fit the data without allowing any slacks. This condition may not always be able to find a solution when the classes are overlapping.

Key Takeaways

- Kernel is used to map data to a higher dimension and make linearly separable.
- Slack variable to penalize the few misclassifications.

- $$\min_{\mathbf{w}, b} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^n \max(1 - y_i (\mathbf{w}^T \mathbf{x}_i + b), 0)$$

$$\text{Regularization, } \mathcal{R}_{\mathbf{w}} = \min_{\mathbf{w}, b} \mathbf{w}^T \mathbf{w}$$

$$\text{Loss, } \mathcal{L}_{\mathbf{w}}(x, y) = C \sum_{i=1}^n \max(1 - y_i (\mathbf{w}^T \mathbf{x}_i + b), 0)$$

- Lower the cost parameter C , Large the margin and vice-versa.
- The Loss is directly proportional to $C \cdot \xi_i$
 - if any of the value of ξ_i increases keeping C constant.
 - if C increases, keeping all the ξ_i constant.

References

- Books
 - Nello Cristianini, John Shawe-Taylor(2000), An Introduction to Support Vector Machines and other Kernel-based Learning Methods, 1st edition
 - Check unit 6, page 107 to understand the Slack Variables for Non-Linearly Separable Problems.
 - Stephen Marsland, Machine Learning An Algorithmic Perspective, 2nd edition
 - Check Chapter 8, page 175 to understand the Slack Variables for Non-Linearly Separable Problems.